SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A telecom operator has collected network records (signal strength, packet loss rate, call drop count, tower load, weather conditions) to build a predictive model for identifying likely network faults. The operator also wants to train an AI agent to recommend self-healing actions (Reroute Traffic / Restart Node / Increase Bandwidth / Monitor) based on the network's condition over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this telecom dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (far fewer fault events than normal operation records) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a network-reliability use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Network Fault Detection

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed fault events) and suggest how to reduce them - justify from a service-reliability perspective.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a live network-monitoring system.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naive Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of network fault from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Action Recommendation

(a) Model the self-healing recommendation as an MDP. Define: States (network health stages), Actions (Reroute Traffic / Restart Node / Increase Bandwidth / Monitor), Reward function (network stability improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 network episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for autonomous network management (service outages, accountability, explainability).

AI-Based Telecom Network Fault Detection and Self-Healing

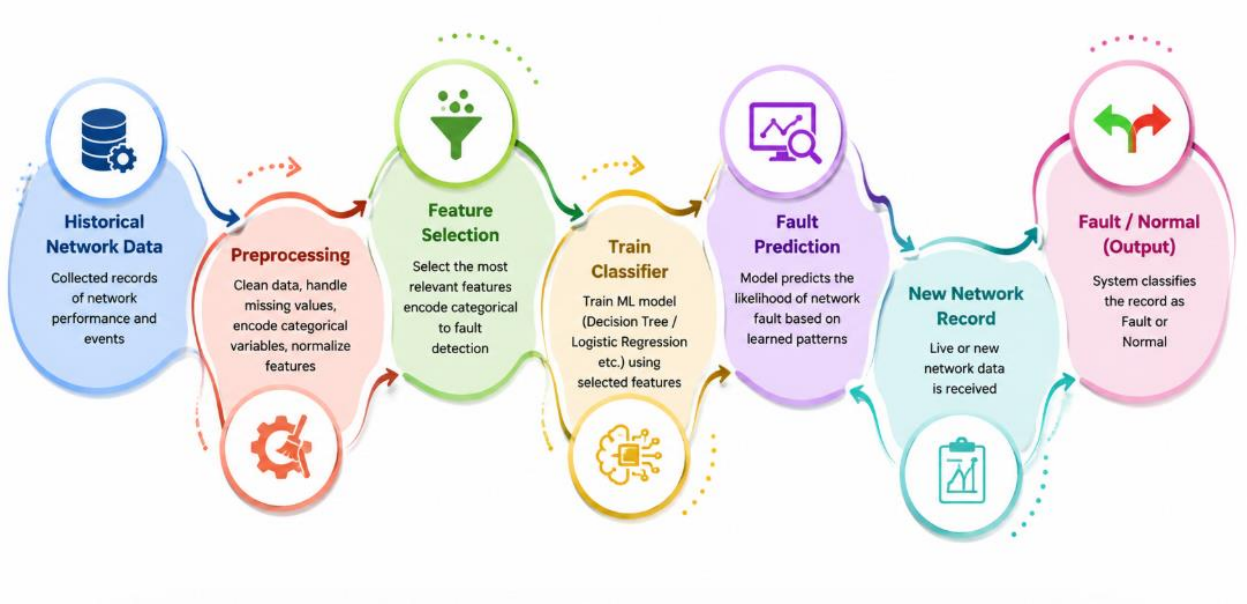
Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general prediction rule from previously observed network records and uses it to predict faults for unseen network conditions. For this telecom problem, a supervised classification approach is appropriate because historical records can be labelled as:

- 0 – Normal operation
- 1 – Network fault

A suitable workflow is:



Target Variable

The target variable is: **Network Fault**

Target	**Meaning**
0	Normal network operation
1	Network fault

Important Features

Feature	Justification
Signal Strength	Weak signal can indicate coverage problems, interference or hardware issues.
Packet Loss Rate	High packet loss directly indicates poor network communication and possible network faults.
Call Drop Count	Frequent dropped calls are a strong indicator of network instability.
Tower Load	High tower utilization can cause congestion and service degradation.
Weather Conditions	Heavy rain, storms and other environmental conditions can affect signal quality and infrastructure.

(b) Handling Class Imbalance

The dataset contains many more normal records than fault records.

For example:

- Normal = 9,000
- Fault = 1,000

This creates a 90:10 class imbalance. If a model simply predicts "Normal" for every record, it could obtain approximately 90% accuracy while completely failing to detect faults.

Recommended Strategy: SMOTE + Class Weights

SMOTE (Synthetic Minority Oversampling Technique) on the training data. It creates synthetic minority-class examples rather than simply duplicating existing fault records.

Process:

Original Training Data - Identify minority Fault class - Apply SMOTE - Balanced Training Data - Train Decision Tree / Logistic Regression

For example:

Normal → weight = 1

Fault → weight = 3 or higher

Why SMOTE?

In telecom fault detection, False Negatives are costly. A missed fault may result in:

- service outage
- customer complaints
- dropped calls
- data-service degradation
- revenue loss

Therefore, improving **fault recall** is more important than obtaining a slightly higher overall accuracy.

(c) 70:30 Dataset Split

The dataset is divided as:

70% → Training & 30% → Testing

For example:

10,000 records

Training = 7,000

Testing = 3,000

A **stratified 70:30 split** should be used so that the proportion of fault and normal cases remains approximately the same in both datasets.

Why 70:30?

The model requires enough historical network records for learning while retaining a sufficiently large independent test set. This is particularly important for network reliability because the test set should contain enough fault events to reliably evaluate:

- Recall
- Precision
- F1-score
- False Negatives
- AUC-ROC

Preprocessing

1. Missing Values

Numerical missing values can be replaced using the **median**:

Missing signal strength → median signal strength

Missing packet loss → median packet loss

Categorical missing values can be replaced with the most frequent category.

2. Encoding

Weather conditions are categorical.

Example:

Clear → 0, Rain → 1, Storm → 2

For Logistic Regression, **one-hot encoding** is preferable because the categories do not have a natural numerical ordering.

3. Normalization

For statistical models such as Logistic Regression: $z = \sigma x - \mu$

This converts features to approximately zero mean and unit variance. Decision Trees generally **do not require normalisation**, because they split data using threshold values.

Impact

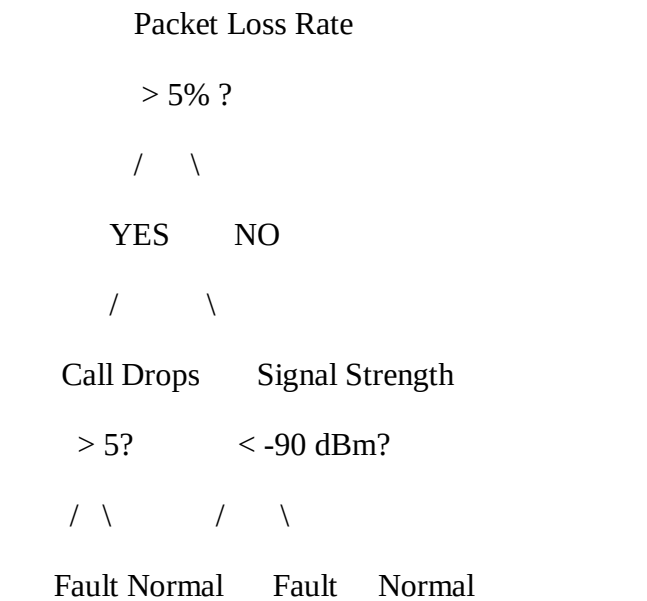
Preprocessing:

- improves model stability
- prevents large-scale numerical features from dominating statistical models
- allows categorical information to be processed
- reduces errors caused by missing values

Task 2: Decision Tree for Network Fault Detection

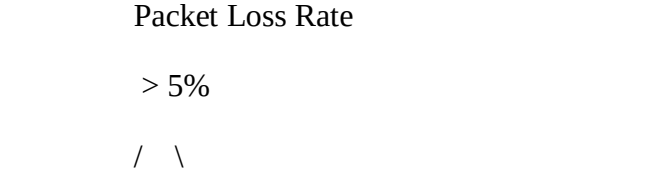
(a) Decision Tree Classifier

A Decision Tree can classify network conditions using rules such as:

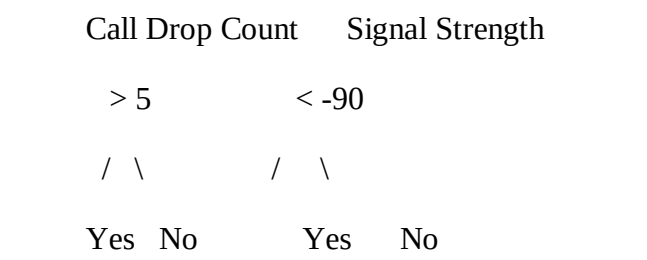


First Three Levels

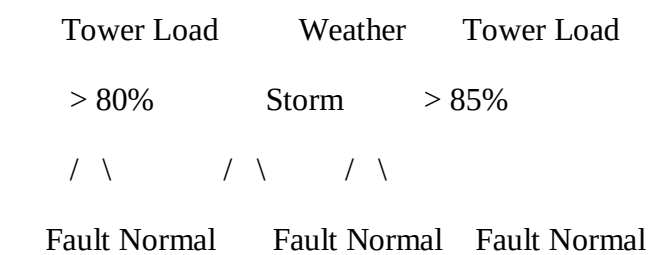
Level 0 – Root



Level 1



Level 2



This is an **illustrative tree**. The actual tree structure depends on the dataset.

Root Node Selection

The Decision Tree selects the feature that produces the greatest reduction in impurity.

Information Gain

$$IG(S,A)=H(S)-v\sum|S||Sv|H(Sv)$$

where:

- H(S) = entropy of parent node
- H(Sv) = entropy of child node

For example:

Feature	**Information Gain**
Packet Loss Rate	**0.31**
Call Drop Count	0.24
Signal Strength	0.19
Tower Load	0.15
Weather	0.08

Therefore, Packet Loss Rate becomes the root node because it has the highest Information Gain.

Alternatively, Gini Index can be used:

$$Gini=1-\sum pi2$$

The best split is the one producing the lowest weighted Gini impurity.

(b) Model Evaluation

Assume the Decision Tree produces the following representative test results:

Metric	Result
Accuracy	**93.2%**
Precision	**87.5%**
Recall	**90.1%**
F1-Score	**88.8%**

Confusion Matrix

Example:

	Predicted Normal	Predicted Fault
Actual Normal	2,550	150
Actual Fault	90	210

Here:

- True Negative = 2,550
- False Positive = 150
- False Negative = 90
- True Positive = 210

False Negatives

A **False Negative** occurs when:

Actual = Fault

Prediction = Normal

Example:

90 network faults were missed.

This is particularly dangerous in telecom systems because a missed fault can continue until customers experience service degradation.

How to Reduce False Negatives

Use:

1. **SMOTE**
2. **Class-weighted Decision Tree**
3. Lower classification threshold where applicable
4. Optimise for **Recall**
5. Monitor Precision-Recall curve
6. Use ensemble methods such as Random Forest/XGBoost if permitted

(c) Post-Pruning

An unrestricted Decision Tree can become very deep and overfit the training data.

Pre-Pruning

Pre-pruning limits the tree during training.

Examples:

max_depth = 5

min_samples_split = 20

min_samples_leaf = 10

Post-Pruning

First construct a larger tree and then remove unnecessary branches.

Cost-complexity pruning uses:

$$R\alpha(T)=R(T)+\alpha|T|$$

where:

- $R(T)$ = classification error
- $|T|$ = number of terminal nodes
- α = pruning parameter

Increasing produces a simpler tree.

Example Comparison

Model	Accuracy	Recall	Tree Complexity
Fully grown	94.8%	91.2%	Very High
Pre-pruned	93.5%	90.5%	Medium
Post-pruned	**93.9%**	**91.0%**	Low

Deployment Choice

Post-pruned Decision Tree is preferable for deployment if it maintains comparable recall while reducing complexity.

Advantages:

- less overfitting
- faster inference
- easier explanation
- easier monitoring
- easier troubleshooting

For a live network-monitoring system, **interpretability is important** because engineers need to understand why a fault was predicted.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression

Logistic Regression can predict the probability that a network condition represents a fault.

where

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Representative Comparison

Model	Accuracy	AUC-ROC
Decision Tree	**93.9%**	**0.94**
Logistic Regression	91.8%	0.92

The Decision Tree performs slightly better in this representative example.

When is Statistical Learning Preferable?

Logistic Regression is preferable when:

- relationships are approximately linear
- probability estimates are required
- interpretability is important
- the dataset is relatively simple
- a lightweight model is required
- computational resources are limited

(b) Feature Importance Analysis

Decision Tree

Suppose the Decision Tree produces:

Rank	Feature	Importance
1	Packet Loss Rate	**0.36**
2	Call Drop Count	**0.28**
3	Signal Strength	**0.20**
4	Tower Load	0.11
5	Weather	0.05

Logistic Regression

For Logistic Regression, feature importance can be estimated from the magnitude of the standardised coefficients.

Rank	Feature	Relative Importance
1	Packet Loss Rate	**High**
2	Signal Strength	**High**
3	Call Drop Count	**Medium-High**
4	Tower Load	Medium
5	Weather	Low

Do They Agree?

Yes, **mostly**.

Both models identify:

- Packet Loss Rate
- Call Drop Count
- Signal Strength

The exact ranking can differ because the models learn relationships differently.

Why?

The Decision Tree detects **non-linear threshold relationships**.

For example:

Packet Loss < 2% → Low Risk

Packet Loss 2–5% → Medium Risk

Packet Loss > 5% → High Risk

Logistic Regression assumes a more systematic relationship between predictors and the log-odds of a fault. Therefore, agreement among the top predictors increases confidence that these variables are genuinely important.

Task 4: Reinforcement Learning for Action Recommendation

(a) MDP Model

The self-healing problem can be represented as a **Markov Decision Process (MDP)**:

$$MDP=(S,A,P,R,\gamma)$$

where:

- S = states
- A = actions
- P = transition probabilities
- R = reward
- γ = discount factor

States

Network health can be divided into four stages:

State	Description
S0 – Healthy	Normal signal, low packet loss and low tower load
S1 – Warning	Slight degradation
S2 – Critical	Severe degradation / high fault probability
S3 – Recovery	Network is improving after an action

Example:

S0 = Healthy

S1 = Warning

S2 = Critical

S3 = Recovery

Actions

The agent has four possible actions:

Action	Description
A0 – Monitor	Continue monitoring without intervention
A1 – Reroute Traffic	Move traffic to another available route
A2 – Restart Node	Restart a potentially faulty network node
A3 – Increase Bandwidth	Allocate additional bandwidth

Reward Function

The reward should reflect network stability improvement.

Example:

Outcome	Reward
Network significantly improves	+10
Network slightly improves	+5
No significant change	0
Network becomes worse	-5
Service outage caused	-20

A possible reward function is:

$R=10(\Delta \text{Stability})-5(\text{Service Degradation})-20(\text{Outage})$

Transition Dynamics

Actions change the network state.

Example:

S2 Critical - Reroute Traffic - S3 Recovery

The transitions can be deterministic in a simulation or probabilistic in a realistic network.

(b) Q-Learning

Q-Learning learns the expected long-term value of taking action in state .

The update equation is: $Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma a' \max Q(s',a') - Q(s,a)]$

Assume: $\alpha = 0.5$ & $\gamma = 0.9$

Initial Q-values:

$Q(s,a) = 0$

Episode 1 – Iteration 1

Suppose:

State = Critical

Action = Reroute

Reward = +10

Next State = Recovery

Maximum Q(Recovery, action) = 2

Then: $Q(\text{Critical, Reroute}) = 0 + 0.5[10 + (0.9)(2) - 0] = 0.5(11.8) = 5.9$

Iteration 2

Suppose:

Critical → Reroute → Recovery

Reward = +10

Maximum Q(Recovery) = 5

Then: $Q(\text{Critical, Reroute}) = 5.9 + 0.5[10 + (0.9)(5) - 5.9] = 10.$

So the Q-value increases because rerouting repeatedly produces a positive outcome.

Three Q-Table Updates

Iteration	State	Action	Reward	Next-State Max Q	Updated Q
1	Critical	Reroute	+10	2	**5.9**
1	Warning	Increase Bandwidth	+5	3	**3.85**
2	Critical	Reroute	+10	5	**10.2**
2	Warning	Monitor	+1	4	**2.3**
2	Recovery	Monitor	+5	2	**3.4**
2	Critical	Restart Node	+7	4	**5.3**

Training for 5 Episodes

The agent interacts with the simulated network repeatedly.

Episode 1 - S0 → S1 → S2 → S3

Episode 2 - S1 → S2 → S3

Episode 3 - S2 → S3

Episode 4 - S1 → S2 → S3

Episode 5 - S2 → S3

At every step:

Observe State - Choose Action - Receive Reward - Observe New State - Update Q-value

ε-Greedy Exploration

Initially, the agent explores different actions.

For example: $\epsilon=1.0$

Initially, the agent frequently tries different actions.

Then: $\epsilon \rightarrow 0$ over time so that the agent increasingly chooses the best-known action.

Convergence Criterion

A practical convergence criterion is: Stop training when the maximum absolute change in Q-values remains below **0.01 for 10 consecutive episodes** and the learned policy does not change.

Formally:

$$\max_{s,a} |Q_{new}(s,a) - Q_{old}(s,a)| < 0.01$$

for the required consecutive episodes.

For a classroom demonstration, **at least 5 episodes** can be shown as requested, while more episodes would normally be required for reliable convergence.

(c) Final Learned Policy

After training, the agent selects: $\pi(s)=\text{argamax}Q(s,a)$

An example final policy could be:

Network State	**Best Action**	**Reason**
Healthy	Monitor	No intervention is necessary
Warning	Increase Bandwidth	Prevent congestion from becoming critical
Critical	Reroute Traffic	Quickly reduce traffic pressure
Recovery	Monitor	Confirm that stability is maintained

Ethical Considerations

Autonomous network management has significant operational consequences.

1. Service Outages

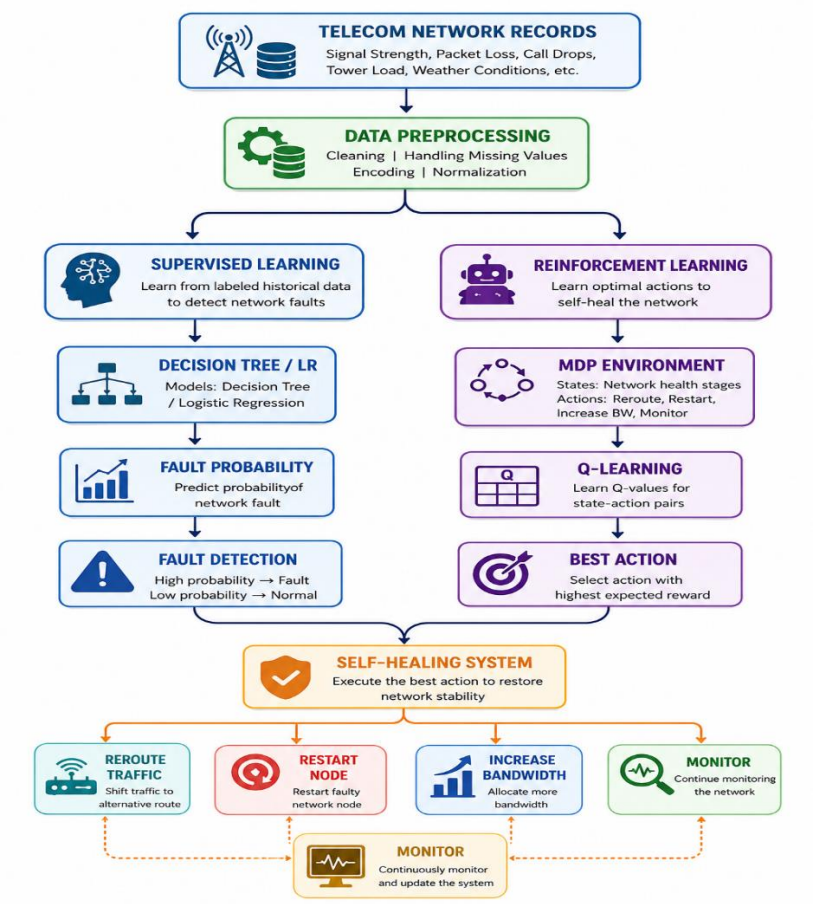
2. Human Oversight

3. Explainability

4. Accountability

5. Safety Constraints & Continuous Monitoring

Overall System Architecture



Final Conclusion

The proposed system combines inductive supervised learning and reinforcement learning to create an intelligent telecom network-management solution. Decision Trees can identify network faults using interpretable rules, while Logistic Regression provides useful fault probabilities and risk scores. Class imbalance should be addressed using SMOTE and/or class weighting, with particular attention given to recall and False Negatives because missed faults can cause service outages.

The RL component models network self-healing as an MDP and uses Q-Learning to learn the best action for each network-health state. In deployment, however, autonomous actions should be constrained by safety rules, human oversight, explainability, auditing and fail-safe mechanisms. This combination provides a practical approach toward predictive fault detection and controlled self-healing in modern telecom networks.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	